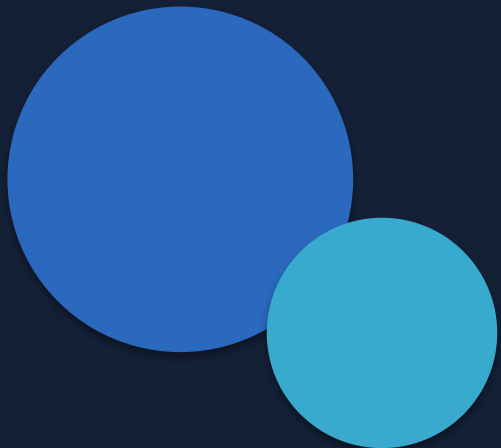


LIBRARY BOOK ISSUE & RETURN MANAGEMENT SYSTEM

DBMS TAE-2 | Live Project Implementation



Project Snapshot

120 members • 120 books
100 staff • 250+ issue
transactions



**Anushka Sharma • Roll No.
P09**

Guided by – Mr. Chinmay

Mukta Raison College of Engineering and Management,
Pune

1. Problem, Domain & TAE-1 Recall

What the system manages — and why a database is needed



The Problem

Library transactions involve multiple entities: members, books, staff and issues.

Manual records make it difficult to track availability, due dates, returns and fines.

Repeated data can cause inconsistency — the same member or book details may appear many times.



The Database Solution

Store each major entity once and connect them using primary and foreign keys.

Use SQL queries to retrieve transaction history, summaries and overdue information.

Automate issue/return effects with procedures and triggers.



TAE-1 Business Rules

One member can borrow many books.

A book can be issued many times over its lifetime, but an active copy must be tracked.

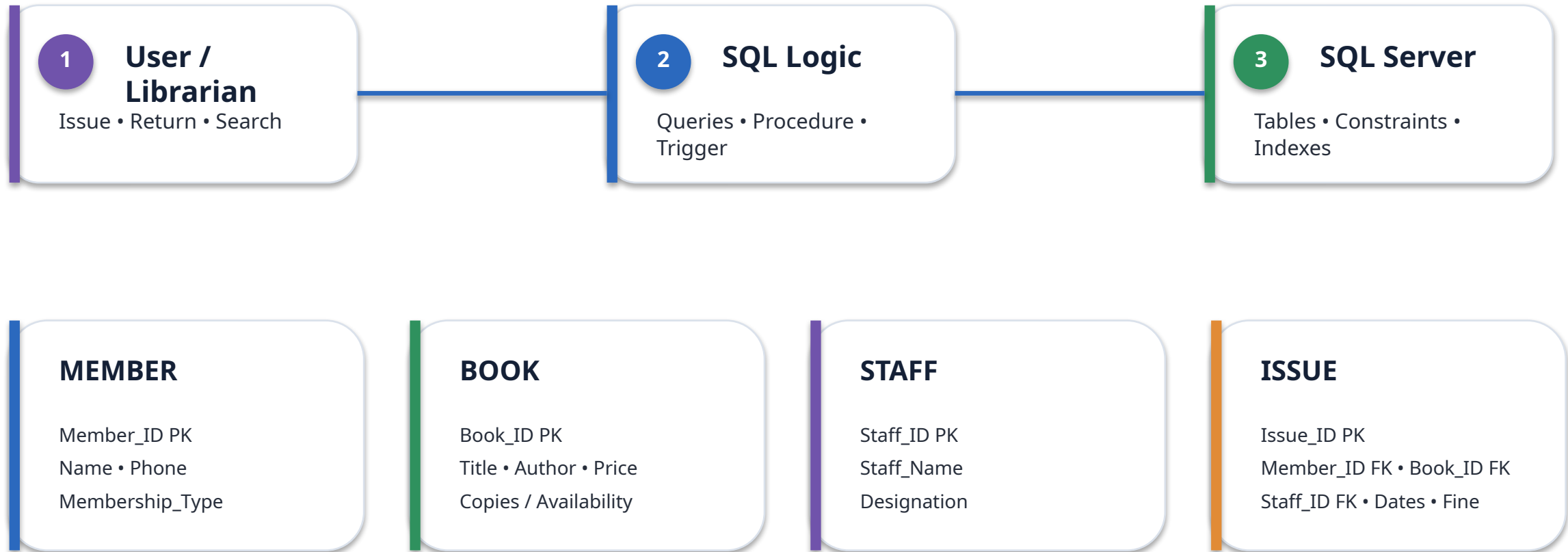
Staff processes issue and return transactions.

Return date and fine amount are recorded.

2. System Architecture & Data Flow

DESIGN

From user action to relational data — simple, traceable and enforceable



3. Relational Schema — Why These Tables?

The schema follows the TAE-1 normalized design and separates responsibilities

TABLE	PRIMARY KEY	IMPORTANT FIELDS	ROLE
MEMBER	Member_ID	Name, Phone, Membership_Type	Stores member identity & membership
BOOK	Book_ID	Title, Author, Price, Availability	Stores catalogue & inventory
STAFF	Staff_ID	Staff_Name, Designation	Identifies transaction handler
ISSUE	Issue_ID	Member_ID, Book_ID, Staff_ID, Dates, Fine	Stores every issue/return event

Why normalization?

Reduces duplication and update anomalies.

Why foreign keys?

Prevent invalid member/book/staff references.

Why constraints?

Move business rules into the database itself.

4. Constraints — Turning Rules into Database Guarantees

Constraints reduce the amount of validation left to application code

PRIMARY KEY

Unique identity for each row.

Example: Member_ID, Book_ID, Staff_ID, Issue_ID

FOREIGN KEY

Maintains valid relationships.

Example: ISSUE → MEMBER / BOOK / STAFF

NOT NULL

Required information cannot be missing.

Example: Member name, phone, etc.

UNIQUE

Prevents duplicate values where required.

Example: Member phone

CHECK

Blocks invalid numeric/business values.

Example: Price $\geq$ 0, Fine $\geq$ 0

DEFAULT

Supplies a safe value automatically.

Example: Fine_Amount = 0

5. Implementation Setup — Why SQL Server / SSMS?

The database engine was changed from the TAE-1 proposal only after instructor approval

W
H

Why I Chose SSMS

SSMS was already installed and available in my development environment.

I already use SQL Server / SSMS for query practice in my AI-DS minor degree.

Using a familiar environment reduced setup time and let me focus on implementation and testing.

The instructor approved proceeding with SQL Server for TAE-2.

S
O

What SQL Server Gives the Project

DDL + DML + constraints for a structured relational database.

Stored procedures and triggers for transaction automation.

Views for reusable reporting logic.

Nonclustered indexes and Actual Execution Plans for performance analysis.

SSMS provides one interface for writing, testing and demonstrating SQL.

INSTRUCTOR-APPROVED IMPLEMENTATION

6. Data Population — Why 100+ Records?

The project is tested with realistic volume instead of only a few sample rows

120

Members

120

Books

100

Staff

250+

Issue records

Why this matters

JOIN queries produce meaningful result sets.

GROUP BY / HAVING and aggregate queries can be demonstrated properly.

Indexes and execution plans can be tested on non-trivial data.

Returned and active issues provide realistic transaction states.

How the data was used

Members are linked to issue transactions.

Books are issued repeatedly across different records.

Staff members process issue transactions.

Return dates and fine values allow reporting and performance tests.

7. Advanced SQL — What I Used & Why

Each SQL feature answers a real reporting or transaction requirement

INNER JOIN

Combine issue + member + book + staff

Result: One complete transaction report

LEFT JOIN

Keep members/books even without matching issues

Result: Find inactive / unused records

SELF JOIN

Compare staff rows within the same table

Result: Group staff with the same designation

GROUP BY + HAVING

Aggregate and filter groups

Result: Members borrowing ≥ 3 books

Subqueries

Compare results against derived averages

Result: Above-average borrowers

Correlated Subquery

Compare against the member's own membership type

Result: Membership-level borrowing analysis

8. Views — Why Save Queries as Views?

Views make frequently used reporting logic reusable and easier to demonstrate

V1 **vw_CurrentlyIssuedBooks**

Purpose: show books that are currently issued.

Combines ISSUE + MEMBER + BOOK + STAFF.

Filters Return_Date IS NULL.

Useful for a librarian who needs the current circulation list.

V2 **vw_MemberBorrowingSummary**

Purpose: summarize borrowing behaviour per member.

Uses LEFT JOIN so members with no issues can still appear.

Uses COUNT / SUM / AVG for borrowing and fine statistics.

Useful for reporting and analysis without rewriting the full query.

2 VIRTUAL VIEWS IMPLEMENTED

9. Stored Procedure & Trigger — Why Automate?

Automation keeps related database changes together and reduces manual steps

PR
O

Stored Procedure: sp_IssueBook

Accepts Member_ID, Book_ID and Staff_ID as parameters.
Checks whether the selected book has available copies.
Creates the ISSUE record with issue date and 14-day due date.
Decreases Available_Copies after a successful issue.
Benefit: one command performs a complete issue transaction.

TR
G

Trigger: trg_BookReturn

Runs automatically after an ISSUE return is recorded.
Detects the transition from Return_Date = NULL to a return date.
Increases Available_Copies in BOOK.
Benefit: inventory update happens automatically when a book is returned.

10. Query Evidence — What Did I Actually Test?

Screenshots in the live demo will show the query, result and execution plan

01 Query 1: Transaction Report

4-table INNER JOIN

ISSUE → MEMBER → BOOK → STAFF

Shows member, book, staff, dates and fine.

Why: demonstrates relational connectivity.

02 Query 2: Borrowing Analysis

GROUP BY + COUNT + HAVING

Finds members with at least 3 issues.

Subquery compares borrowers against an average.

Why: demonstrates analytical SQL.

03 Query 3: Focused Index Test

WHERE Book_ID = 8

Wide SELECT may use a clustered scan on a small table.

Focused SELECT Book_ID uses IX_ISSUE_Book_ID with an Index Seek.

Why: shows optimizer chooses access path based on cost.

11. Performance Tuning — Why Indexes?

Indexes can reduce the work needed to locate rows, but SQL Server still chooses the lowest-cost plan

ID
X

Indexes Present

IX_ISSUE_Member_ID
IX_ISSUE_Book_ID
IX_ISSUE_Staff_ID
Plus clustered primary-key index.

PL
A

Observed Plan

Wide query selecting many ISSUE columns → Clustered Index Scan.
Focused query selecting only Book_ID → Nonclustered Index Seek.
Logical reads were low on the current dataset.

W
H

What This Shows

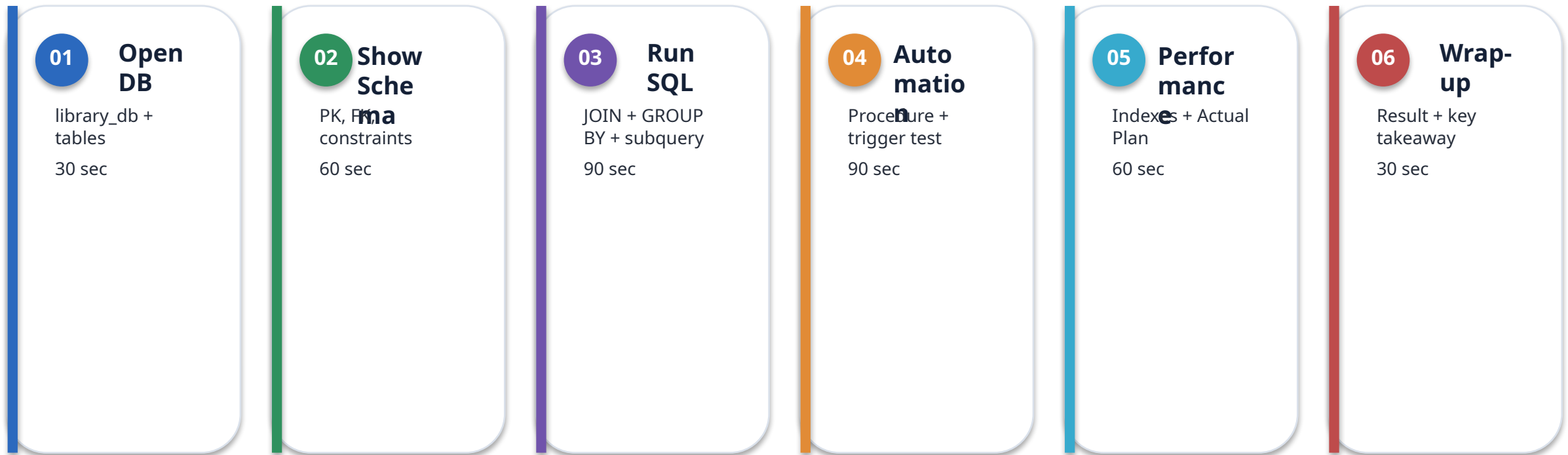
An index does not guarantee an Index Seek.
SQL Server considers table size, estimated cost and requested columns.
The test is evidence of cost-based optimization, not a claim that every query becomes faster.

IMPORTANT: DO NOT CLAIM A FABRICATED BEFORE/AFTER RESULT

12. Live Demonstration — 6 Minutes

DEMO

A clean order prevents the demo from becoming a code dump



- Show result first, then explain why the SQL was chosen.
- Keep full code in SQL files; use the presentation for the logic and evidence.

13. Viva Preparation — Questions I Can Defend

Answer in the pattern: definition → why used here → project example

Why SQL Server?

Existing SSMS environment + prior SQL Server practice + instructor approval.

Why primary / foreign keys?

Unique identity and referential integrity between tables.

Why a stored procedure?

Encapsulates the issue transaction and accepts parameters.

Why a trigger?

Automatically updates inventory when a return occurs.

Why indexes?

Reduce search work for frequently filtered columns.

Why can SQL Server scan instead of seek?

The optimizer compares estimated costs; small tables or wide column retrieval can favour a scan.

What is ACID?

Atomicity, Consistency, Isolation and Durability — properties of reliable transactions.

What is concurrency?

Handling multiple transactions without producing inconsistent results.

WHAT THIS PROJECT DEMONSTRATES

A relational database that is structured, constrained, queryable and partially automated.

STRUCTURE

Normalized tables + PK/FK relationships

QUERY

Joins + aggregates + subqueries + views

AUTOMATION

Procedure + trigger

PERFORMANCE

Indexes + execution-plan analysis

Thank you • Questions?